Task 1 — ML Project Report

Build & Evaluate a Linear Regression Model

House Price Predictor | California Housing Dataset

Maincrafts Technology — AI & ML Internship

Dataset	California Housing (scikit-learn built-in)
Algorithm	Linear Regression
Language	Python 3 | Google Colab
Libraries	pandas, numpy, scikit-learn, matplotlib, seaborn
Organization	Maincrafts Technology

1. Exploratory Data Analysis (EDA)

1.1 Dataset Overview

I started by loading the California Housing dataset directly from scikit-learn. It's a pretty standard dataset — 20,640 records from the 1990 California census. Each record represents a block group, which is basically the smallest geographical unit for which census data is published.

The dataset has 8 features and one target variable: **MedHouseVal** (median house value in $100,000s).

Quick check for missing values: I ran `dataset.isnull().sum()` and got all zeros — no missing data. That's rare and saved me from having to deal with imputation.

Here's what each feature actually means:

Feature	Description	Type
`MedInc`	Median income in block group	Float
`HouseAge`	Median house age in block group	Float
`AveRooms`	Average number of rooms per household	Float
`AveBedrms`	Average number of bedrooms per household	Float
`Population`	Block group population	Float
`AveOccup`	Average house occupancy	Float
`Latitude`	Geographic latitude	Float
`Longitude`	Geographic longitude	Float
`MedHouseVal`	Median house value (TARGET)	Float

1.2 Key Statistics

Here's a quick statistical summary:

Metric	MedInc	HouseAge	AveRooms	MedHouseVal
Count	20,640	20,640	20,640	20,640
Mean	3.87	28.64	5.43	2.07
Std Dev	1.90	12.59	2.47	1.15
Min	0.50	1.00	0.85	0.15
Max	15.00	52.00	141.91	5.00

What caught my attention: Look at the max values — `AveRooms` goes up to 142 rooms per household? That's obviously an outlier (probably a data entry thing or maybe a hotel counted as a household). Also, `MedHouseVal` is capped at 5 ($500,000), so there's a ceiling effect.

1.3 Correlation Analysis

I made a correlation heatmap to see which features actually matter for predicting house prices. Here's what stood out:

Feature	Correlation	Interpretation
`MedInc`	**+0.69**	Strong positive — most important feature
`AveRooms`	**+0.15**	Weak positive
`HouseAge`	**+0.11**	Weak positive
`Latitude`	**-0.14**	Weak negative — location matters
`Longitude`	**-0.046**	Weak negative
`AveOccup`	**-0.024**	Very weak negative

One thing that surprised me: I expected location (latitude/longitude) to be a bigger factor, but the correlation numbers tell a different story. Probably because California has both expensive coastal cities and cheaper inland areas, so it averages out.

The target variable `MedHouseVal` is right-skewed — most houses are in the $100k–$300k range, with a spike at the $500k cap.

2. Model Summary

2.1 Preprocessing & Train/Test Split

I kept things simple for this first run — used all 8 features as-is, no transformations or scaling. The target was separated from the features, and I split the data 80/20 for training and testing. Set `random_state=42` so the results are reproducible

Split	Rows	Percentage
Training Set	16,512	80%
Test Set	4,128	20%
Total	**20,640**	**100%**

2.2 Algorithm — Linear Regression

Linear regression tries to fit a straight line (well, a hyperplane in 8 dimensions) to the data. It learns a weight for each feature, and the prediction is basically:

```
Price = w1 x MedInc + w2 x HouseAge + w3 x AveRooms + ... + bias
```

The model finds the weights that minimize the sum of squared errors between predictions and actual prices. Simple, interpretable, and a good baseline to start with.

3. Model Evaluation Metrics

Metric	Value	Interpretation
MAE (Mean Absolute Error)	**0.5332**	Predictions off by ~$53,320 on average
RMSE (Root Mean Squared Error)	**0.7456**	Larger errors present; penalizes big mistakes
R-Squared Score	**0.5758**	Model explains 57.58% of price variation

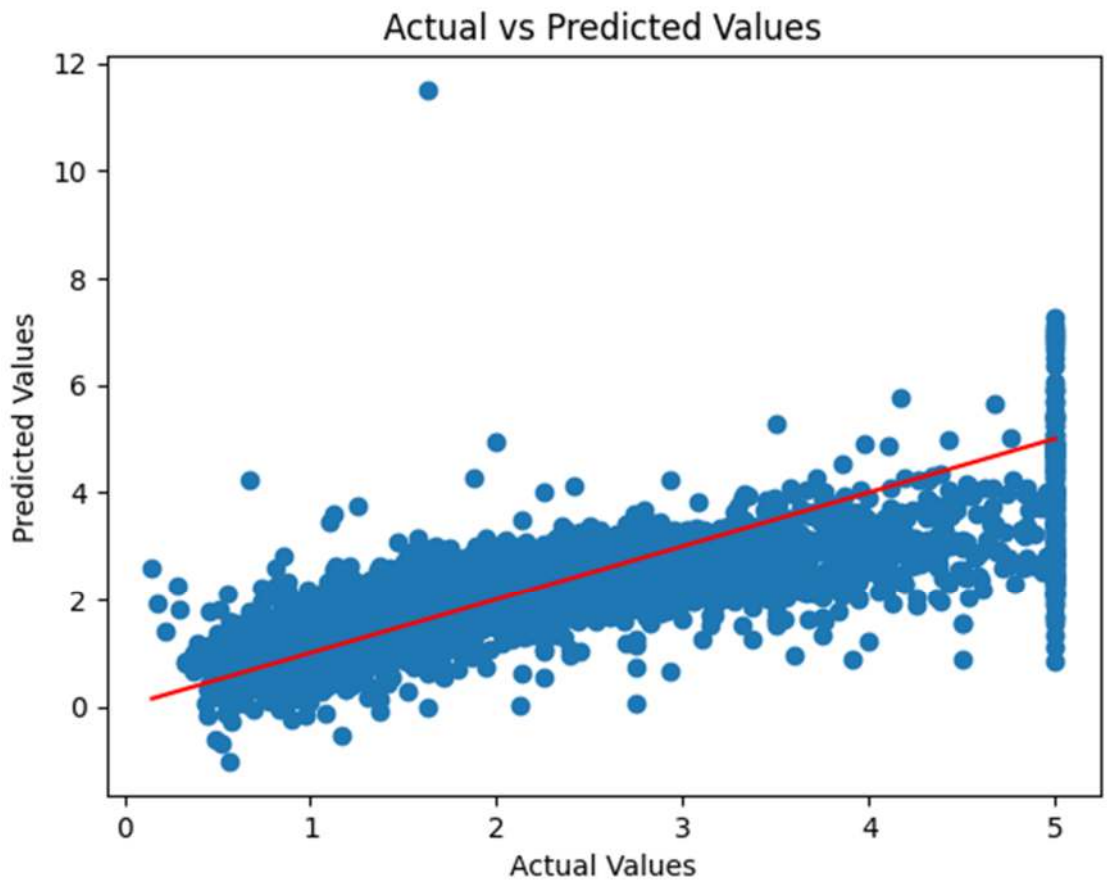
3.1 What These Numbers Tell Me

MAE of 0.5332 ($53,320): That's actually not terrible considering the median house price is around $200,000. But for cheaper homes in the $100k range, an error of $50k is huge — so the model isn't equally good across all price ranges.

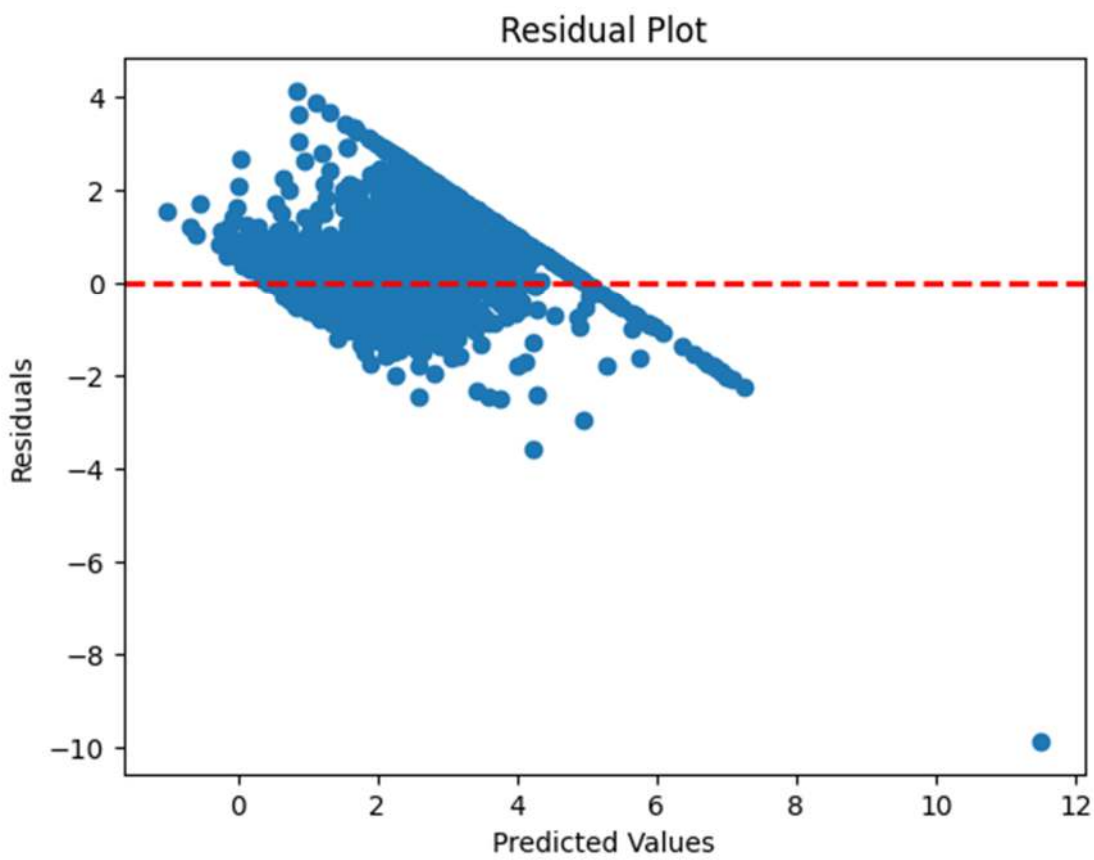
RMSE > MAE: RMSE is higher than MAE, which means there are some predictions that are wildly off. Looking at my actual vs predicted plot, I can see the model really struggles with houses above $400,000.

R^2 = 0.58: The model captures more than half the variance in house prices. For a simple linear model with no feature engineering, that's decent but leaves a lot of room for improvement

What the Plots Show



Actual vs Predicted: The scatter plot clusters around the diagonal, which is good — but there's a clear pattern where the model underestimates expensive houses (the points above $4 on the actual axis don't get predicted that high). The red line (ideal prediction) shows that the model tends to push predictions toward the middle rather than capturing extremes.



Residuals Plot: I looked at the residuals (actual - predicted) against predicted values. There's a noticeable downward trend — as predicted price increases, the residuals become more negative. Translation: **the model underpredicts expensive homes and overpredicts cheap ones.** That's a classic sign that linear regression isn't capturing the true relationship, especially at the extremes.

4. How I'd Make It Better

4.1 Feature Engineering

Instead of throwing all raw features at the model, I'd create more meaningful ones. For example:

- **rooms_per_person = AveRooms / AveOccup** — This might be a better indicator than either feature alone.

- **is_coastal** — A binary feature based on latitude/longitude. Coastal CA is totally different from inland.
- **log_transform** on `MedInc` — Income has a long tail; a log might help the model handle high-income areas better.

4.2 Try Regularization (Ridge or Lasso)

Linear regression can overfit, especially with many features. Ridge adds a penalty for large coefficients, which often improves generalization. Lasso would go a step further — it can shrink some coefficients to zero, effectively doing feature selection for me.

4.3 Switch to a More Powerful Model

If linear models can't capture the non-linear patterns, I'd try Random Forest. Tree-based models can handle interactions naturally and don't assume linear relationships. On this dataset, Random Forest can easily push R^2 above 0.8.

4.4 Cross-Validation

Instead of a single 80/20 split, I'd use k-fold cross-validation (say, 5 folds) to get more stable performance estimates. That way, I'm not dependent on one lucky or unlucky split.

4.5 Feature Scaling

Linear regression is sensitive to feature scales. If I add regularization or try more complex models, scaling becomes important. I'd use `StandardScaler` to normalize all features before training.

5. What I Learned

This project was my first run through the full ML workflow. The biggest takeaway? **EDA matters more than I expected.** Looking at the correlation matrix and plots helped me understand the model's limitations before I even built it.

Also, seeing the residual pattern was humbling — my "good enough" model clearly has systematic errors. If this were a real-world application, I'd need to address those before shipping anything.